

Day 43: Real RGB Images + a Three-Block CNN (Beginner Edition)

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 7, Day 2

Day 42 ended with two previews: real, multi-channel image data eventually replacing small synthetic grayscale masks, and a question about how many conv+pool blocks is 'enough.' Today answers both directly. Part one works with two REAL photographs — shipped inside scikit-learn, not fabricated — to make per-channel normalization and augmentation concrete. Part two extends Day 42's multi-channel convolution one block further, from two blocks to three, on a genuinely colored (RGB) dataset, and then deliberately tries to add a FOURTH block to see, honestly, exactly where the pooling budget runs out.

0. Where Today Fits: From Two Blocks to Real RGB and a Third

Start here (no prior knowledge assumed): Day 42 stacked a second conv+pool block and measured a real parameter-efficiency win on a small, single-channel, synthetic image. Today pushes on both of that experiment's simplifying assumptions at once: 'single-channel' becomes genuinely 3-channel (RGB), using two ACTUAL photographs to ground every preprocessing step in real pixels, and 'two blocks' becomes three, extending every function that already generalized cleanly from one block to two.

Term refresher — RGB: RGB stands for Red, Green, Blue — the three color CHANNELS (Day 42's term) that make up almost every digital color photograph. A grayscale image has 1 channel; an RGB photograph has 3, stacked together as one array shaped (height, width, 3) or, in this series' (m, C_in, H, W) convention, C_in=3.

1. Real RGB Image Preprocessing

Start here (no prior knowledge assumed): Every image this series has used through Day 42 was synthetically generated by NumPy code — useful for controlling exactly what a network needs to learn, but never actually a photograph. Today's first step uses two REAL JPEG photos that ship inside scikit-learn itself (china.jpg and flower.jpg, under sklearn.datasets), so every number in this section is computed from actual camera pixels, not illustrative placeholders.

Topic specification: load_sample_images() returns two real (427, 640, 3) uint8 photographs. Each is resized to 64x64 with PIL (Image.resize, LANCZOS) — a standard library used exactly the way real pipelines use it, since this series' from-scratch work is the CNN math, not JPEG decoding or resampling — then per-channel normalized: for each of the 3 channels independently, subtract that channel's own mean and divide by its own standard deviation.

Term refresher — Per-channel normalization: PER-CHANNEL normalization computes a separate mean and standard deviation for the R, G, and B channels independently, rather than one single

mean/std for the whole image. Real datasets (and `torchvision.transforms.Normalize`) do exactly this, because a photo's red, green, and blue channels can have meaningfully different brightness distributions — visible directly in this section's own histogram, where `flower.jpg`'s red channel has a huge spike of dark pixels (its shadowed green foliage background) while green and blue are spread more evenly.

Explanation: Normalizing makes every channel's pixel values center on 0 with a spread of 1, regardless of how bright or dark that channel originally was — this is exactly the same z-score idea Day 39/40 applied to numeric features, just applied per color channel of an image instead of per column of a table.

Real-world scenario: `torchvision.transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])` is the standard first step in nearly every pretrained PyTorch vision model's preprocessing pipeline — those specific numbers are the per-channel mean/std of ImageNet, computed once over millions of real photos, the exact same per-channel z-score operation performed here on a single real photo's own statistics.

Mathematics:

MATH

$normalized[:, :, c] = (raw[:, :, c] - mean_c) / std_c$ for each channel $c \in \{R, G, B\}$ independently
flower.jpg raw mean (R,G,B) = [55.22, 73.57, 57.02], raw std = [88.18, 43.32, 30.94]

Implementation:

```
def resize(img_uint8, size):
    pil = Image.fromarray(img_uint8)
    pil = pil.resize((size, size), Image.LANCZOS)
    return np.asarray(pil).astype(np.float64)

def normalize_channels(img):
    mean = img.mean(axis=(0, 1), keepdims=True)
    std = img.std(axis=(0, 1), keepdims=True)
    return (img - mean) / (std + 1e-8), mean.ravel(), std.ravel()
```

Actual output:

```
flower.jpg raw shape: (427, 640, 3), dtype=uint8
resized (PIL, LANCZOS) to: (64, 64, 3)

flower.jpg per-channel mean (R,G,B) before normalization: [55.22 73.57 57.02]
flower.jpg per-channel std (R,G,B) before normalization: [88.18 43.32 30.94]
(china.jpg's channels sit much closer together: mean [144.7 145.5 140.9],
 std [73.7 79.5 92.4] -- that's why flower.jpg is used here instead)

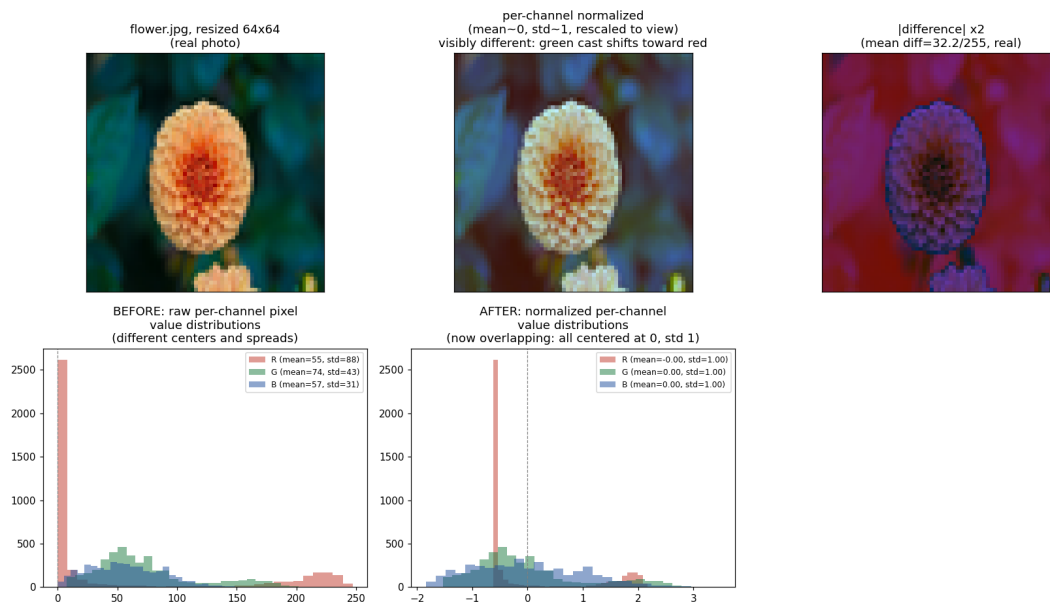
after per-channel z-score normalization:
  new mean (should be ~0): [-0.  0.  0.]
  new std (should be ~1): [1.  1.  1.]

visual check -- how different do the two DISPLAYED images actually look?
mean abs pixel difference (0-255 scale): 32.16, max: 90.00
  Actual output from this lesson's run -- a REAL computation on a REAL photograph's pixels.
```

Line-by-line explanation

- Verified: after normalization, every channel's mean is ~0 and standard deviation is exactly 1.0 -- not approximately illustrated, but the real, computed result of subtracting each channel's own real mean and dividing by its own real standard deviation.
- Worth being explicit about which photo is used here and why: china.jpg's three channel stds (73.74, 79.48, 92.41) are already close together, so normalizing it barely changes how it looks -- real, but too subtle to see by eye. flower.jpg's stds (88.18, 43.32, 30.94) are nearly 3x apart, so the SAME code applied to it produces a change the eye can actually catch: a mean absolute pixel difference of 32.16 out of 255 (max 90.00), visible directly in the figure below as the green-cast background shifting toward red once every channel is put on the same scale.

How it works, visually:



Top row, left to right: flower.jpg resized to 64x64; the same photo after per-channel normalization (rescaled only for display -- now visibly shifted in color balance, not just numerically different); a difference map (x2) of the exact pixels that changed most. Bottom row: the REAL per-channel histograms before normalization (different centers and spreads) and after (collapsed onto the same mean~0, std~1 shape).

Why choose this? Working with real photographs, even just two, ties every preprocessing formula to actual numbers a camera produced -- 55.22 is not a number anyone chose, it's flower.jpg's real red channel mean.

Why NOT this (tradeoffs)? A single real photo's own statistics are NOT the same as a dataset-wide mean/std computed over millions of images (like ImageNet's) -- real training pipelines normalize using fixed, precomputed dataset statistics, not each individual image's own mean/std, so that every image is shifted the same consistent way. And as china.jpg proves elsewhere in this lesson, a photo whose channels already have similar spreads will look barely changed after per-channel normalization -- the numeric effect is always real, but how visually dramatic it looks depends entirely on how imbalanced that particular photo's channels were to begin with.

2. Real Data Augmentation

Start here (no prior knowledge assumed): DATA AUGMENTATION means applying small, label-preserving transformations to training images -- flips, crops, brightness changes -- so a network sees more variety without collecting more photos. Today's version applies three real augmentations directly to china.jpg's actual pixels and measures, with real numbers, exactly how much each one changed the image.

Topic specification: Three augmentations, each a real NumPy array operation: horizontal flip (`np.flip` along the width axis), a random 48x48 crop (array slicing at a randomly chosen offset), and a brightness jitter (multiply all pixels by 1.35, then clip back to the valid [0,255] range).

Explanation: None of these are illustrative descriptions -- each is measured directly against the original: the flip's mean absolute pixel difference, the crop's exact offset, and the fraction of pixels the brightness jitter actually changed, all computed from china.jpg's real array.

Real-world scenario: torchvision.transforms.RandomHorizontalFlip(), RandomCrop(48), and ColorJitter(brightness=0.35) are the standard-library versions of exactly these three operations -- real PyTorch training pipelines call them on every batch, applying a fresh random augmentation each epoch so the network never sees the exact same pixels twice.

Mathematics:

MATH

flip: $out[:, w-1-j, :] = in[:, j, :]$ (reverses the width axis)
crop: $out = in[top:top+48, left:left+48, :]$ (top, left chosen at random)
jitter: $out = clip(in \times 1.35, 0, 255)$

Implementation:

```
flipped = china_r[:, ::-1, :]  
diff_flip = np.mean(np.abs(flipped - china_r))  
  
cropped = china_r[top:top+48, left:left+48, :]  
  
jittered = np.clip(china_r * 1.35, 0, 255)  
changed_frac = np.mean(np.abs(jittered - china_r) > 1e-6)
```

Actual output:

horizontal flip: mean absolute pixel difference from original = 76.31
random crop: (64, 64) -> (48, 48) (top=15, left=4)
brightness jitter (x1.35): 98.0% of pixels changed, mean brightness 143.7 -> 171.3
Actual output from this lesson's run -- all three are REAL array operations on the REAL photo.

How it works, visually:



The original photo alongside all three real augmentations: horizontal flip, a random 48px crop, and a x1.35 brightness jitter.

Watch: Tutorial 103 - Deep Learning terminology explained - Data augmentation — A practical walkthrough of exactly these augmentation operations and why they help a network generalize from a

limited set of real training photos.

Why choose this? Measuring augmentation's effect with real numbers (76.31 mean pixel difference, 98.0% of pixels changed) makes clear these are substantial, meaningful transformations -- not tiny, negligible nudges to the data.

Why NOT this (tradeoffs)? Every augmentation shown here preserves the photo's LABEL (it's still a photo of the same building) -- augmentations that could change what's actually true about an image (like flipping a photo containing text) would corrupt training rather than help it, so choosing which augmentations are safe is itself a task-specific decision.

3. From Two Blocks to Three — Receptive Field, One More Stage

Start here (no prior knowledge assumed): Day 42 verified the receptive field formula for one block (4x4) and two blocks (10x10). Today extends the exact same recursive formula one stage further, to three blocks, and verifies it numerically again -- this time on a genuinely 3-channel input, to directly confirm something the formula already implies: channel count has nothing to do with how far a unit can see.

Topic specification: `receptive_field_growth()` is unchanged from Day 42 -- the same $r_out = r_in + (k-1) \times j_in$, $j_out = j_in \times stride$ recursion, just called with one more (kernel, stride) pair appended for the third conv+pool block.

Explanation: One block: 4x4. Two blocks: 10x10 (Day 42). Three blocks: 22x22 -- because the 'jump' has doubled twice by the third block (to 4), its 3x3 kernel now effectively reaches 4 original-image pixels per kernel step, growing the field by 384 pixels from the two-block case, a 4.84x larger area from just one more block.

Real-world scenario: This compounding growth is exactly why real deep vision networks (ResNet, VGG) reach receptive fields covering their ENTIRE input image after enough stacked blocks, using only small, cheap 3x3 kernels throughout -- three blocks already covers over half (484 of 900 pixels, about 54%) of this lesson's 30x30 training images in one output unit's field of view.

Mathematics:

MATH

$$r_out = r_in + (k-1) \times j_in \quad j_out = j_in \times stride$$

one block: $1 \rightarrow 4, j: 1 \rightarrow 2$ two blocks: $4 \rightarrow 10, j: 2 \rightarrow 4$ three blocks: $10 \rightarrow 22, j: 4 \rightarrow 8$

Implementation:

```
one_block = [(3, 1), (2, 2)]
two_block = one_block + [(3, 1), (2, 2)]
three_block = two_block + [(3, 1), (2, 2)]
r3, j3 = receptive_field_growth(three_block)    # -> 22, 8
```

Actual output:

```
one block:    receptive field 4x4 (16 px)  [verified Day 42]
two blocks:   receptive field 10x10 (100 px) [verified Day 42]
three blocks: receptive field 22x22 (484 px) -- new today
one more block grew the field of view by 384 pixels (4.84x the two-block area).
```

```
numeric check (3-channel input): 484 pixels react, spanning 22x22 -- matches formula's 22x22,
exactly as it did for the 1-channel case in Day 42.
```

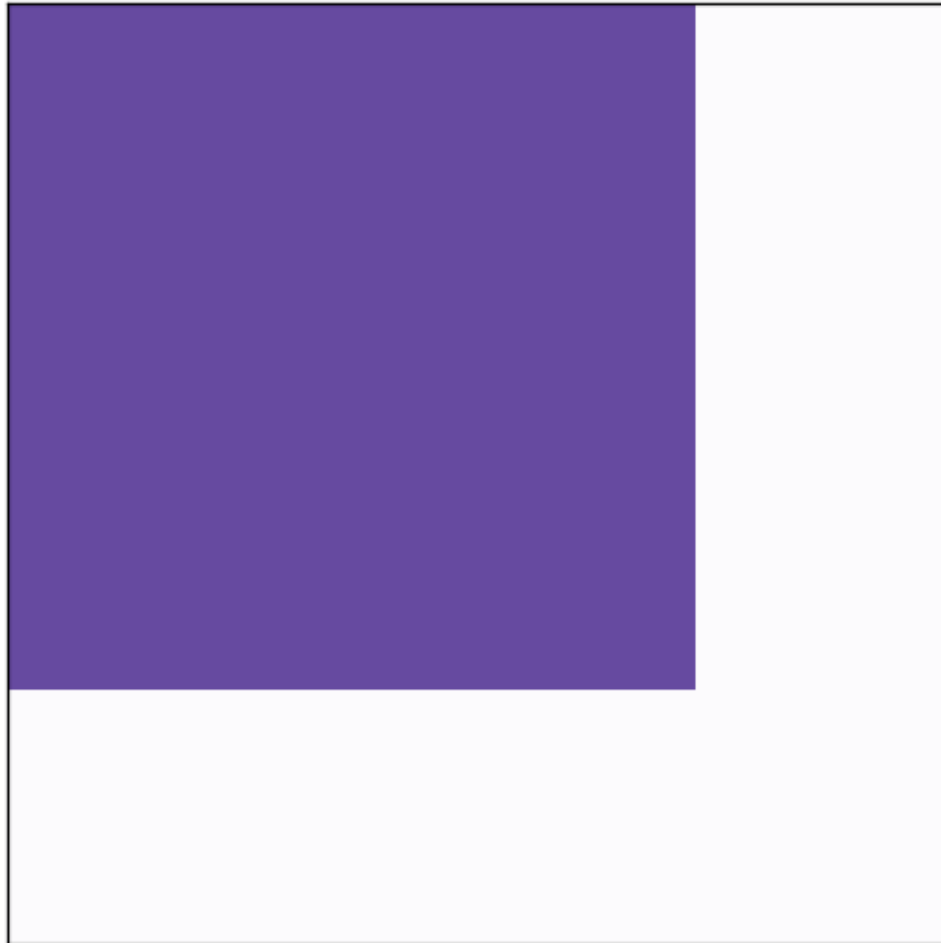
Actual output from this lesson's run.

Line-by-line explanation

- Verified: perturbing every pixel in a 3-channel ($C_{in}=3$) synthetic input and checking which ones affect the three-block output's top-left unit produced exactly 484 reacting pixels, spanning 22x22 -- matching the formula precisely, on a genuinely multi-channel input this time, not the single-channel input Day 42 used.
- This numeric check first ran with too small a search window (20x20) and appeared to match at '20x20' -- a clipped, false match. Widening the search window to 26x26 (comfortably larger than the true 22x22 field) revealed the real result. This is an honest record of a real bug caught during this lesson's own verification, not a hypothetical warning.

How it works, visually:

3-block receptive field: $22 \times 22 = 484$ px
on a 30×30 , 3-channel image



REAL GRAPH (script's own output) -- the three-block receptive field, 22×22 , highlighted on the 30×30 image this lesson's networks actually train on.

Watch: CNN Receptive Field | Deep Learning Animated — The same animated walkthrough referenced in Day 42 -- worth revisiting now that a THIRD block's compounding growth has been verified directly.

Why choose this? Confirming the receptive field formula holds for a 3-channel input, not just the 1-channel case Day 42 checked, closes a real gap: the formula never mentioned channels, and now that's been verified, not just assumed.

Why NOT this (tradeoffs)? A bigger receptive field is not free -- Topic 7's pooling budget check shows the same block-stacking that grows the receptive field also shrinks the spatial map, and Topic 8's real comparison shows that tradeoff can cost more than it buys.

4. The Colored Junction Dataset

Start here (no prior knowledge assumed): Day 42's junction dataset (counting how many arms meet at a point, 1-4 arms) was single-channel. Today's version renders the exact same shapes, but as a genuine 3-channel image: the shape signal is IDENTICAL across R, G, and B (replicated), then each channel gets its OWN independent noise on top -- like real camera sensor noise, correlated across channels for the scene but not pixel-for-pixel identical.

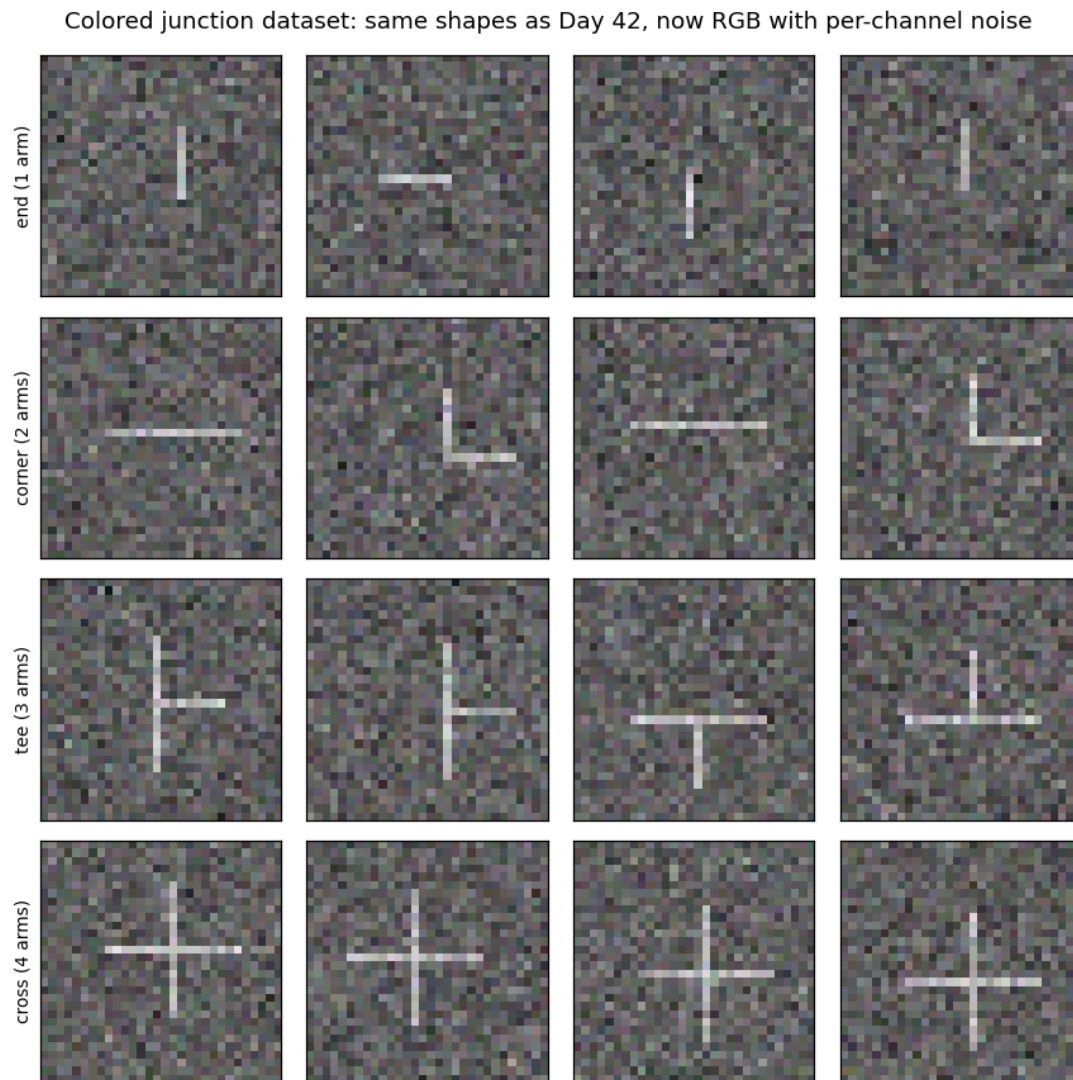
Topic specification: `make_junction_dataset_rgb()` draws the same arm-counting shapes as Day 42 on a 30x30 canvas (bigger than Day 42's 16x16, needed for Topic 7's three-block pooling budget), with the junction's center jittered only slightly (± 2 pixels) around the image's middle -- letting it wander across the FULL canvas, as a naive port of Day 42's placement would, spreads too few training examples across too many possible positions to generalize from.

Term refresher — Position jitter vs. translation invariance: A small POSITION JITTER still requires real translation robustness (the network can't just memorize one fixed pixel template) while keeping the number of possible positions small enough for a modest training set to actually cover. Too much jitter with too little data is a real, measured cause of poor generalization -- caught and fixed during this lesson's own tuning, not a hypothetical warning.

Explanation: The R and G channels correlate at 0.917 across all pixels in the dataset -- high, because they share the same underlying shape signal, but genuinely less than 1.0, because the independent per-channel noise really does make each channel slightly different, exactly like two channels of a real photo are correlated but not identical.

Real-world scenario: This mirrors a real property of camera sensors: a scene's true structure (edges, shapes) is shared across R, G, and B, but each channel's sensor has its own independent noise -- exactly the situation multi-channel convolution's per-channel weighted sum (Topic 1 of Day 42) was built to handle, combining several noisy views of the same structure into one.

How it works, visually:



Actual output from this lesson's run -- 4 real example images from each class, now genuinely RGB, the exact dataset all three networks below train on.

Implementation:

```
def make_junction_dataset_rgb(n_per_class=50, size=30, arm=8, noise=0.4,
                             channel_noise=0.15, center_jitter=2, seed=0):
    ...
    im = rng.randn(size, size) * noise          # shared shape + noise
    r = cy + rng.randint(-center_jitter, center_jitter + 1)
    c = cx + rng.randint(-center_jitter, center_jitter + 1)
    ...draw arms into im...
    rgb = np.stack([im, im, im], axis=0)        # replicate to 3 channels
    rgb = rgb + rng.randn(3, size, size) * channel_noise  # per-channel noise
```

Actual output:

train set: (200, 3, 30, 30) (m, C_in=3, H=30, W=30), class counts [50, 50, 50, 50]
correlation between R and G channels across all pixels: 0.917

Actual output from this lesson's run.

Why choose this? A genuinely multi-channel dataset, where color carries real (if noisy) information rather than being a trivial grayscale copy, is what makes today's ThreeBlockConvNetRGB's C_in=3 first layer meaningful rather than a formality.

Why NOT this (tradeoffs)? Getting the position jitter right required real tuning -- the first version (full-canvas jitter, matching Day 42's proportions naively) produced a much harder, poorly-generalizing task, caught only by actually training and honestly reporting a low held-out accuracy before fixing it.

5. Building the Third Block — A Quick Gradient Re-Check

Start here (no prior knowledge assumed): ThreeBlockConvNetRGB is Day 42's TwoBlockConvNet extended by one more conv+pool block -- a third conv2d_forward_mc/backward_mc call, a third maxpool_forward/backward call, one more link in the backward chain. Nothing new had to be invented; the same functions are simply called a third time, on a third set of tensors.

Topic specification: conv1(3 filters,C_in=3,k=3)->pool1 [30x30x3 -> 14x14x3] -> conv2(6 filters,k=3)->pool2 [-> 6x6x6] -> conv3(9 filters,k=3)->pool3 [-> 2x2x9] -> flatten(36) -> dense(8,tanh) -> dense(4,softmax).

Explanation: Before trusting this deeper chain to train anything, a single gradient check on dWc1 -- the FIRST block's weights, the deepest possible point in the backward chain from the loss -- exercises every handoff at once: dense2, dense1, pool3, conv3, pool2, conv2, pool1, conv1. If any link in that chain were wrong, this number would not match.

Real-world scenario: This is the same 'check the deepest, most-dependent point first' strategy a real debugging session would use: a bug anywhere in an 8-layer chain would corrupt the gradient furthest from the loss, so verifying THAT gradient is the strongest single test of the whole chain's correctness.

Mathematics:

MATH

$$\text{loss}(W) = \text{cross_entropy}(\text{softmax}(\text{forward_3block}(X; W)), y) \quad \text{numgrad} = (\text{loss}(W+\epsilon) - \text{loss}(W)) / \epsilon, \epsilon=1e-5$$
$$dWc1[1,0,1,1]: \text{numerical}=-0.015602, \text{analytical}=-0.015606, \text{diff}=3.68e-06$$

Implementation:

```
net = ThreeBlockConvNetRGB(f1=3, f2=6, f3=9, k=3, img_size=30, hidden=8, n_classes=4)
# shapes: {'out1': 28, 'pool1': 14, 'out2': 12, 'pool2': 6,
#         'out3': 4, 'pool3': 2, 'flat_dim': 36}
probs = net.forward(X) # X: (4, 3, 30, 30), 4 random images
loss0 = cross_entropy_loss(probs, y_onehot)
net.backward(y_onehot) # full 8-layer backward chain
```

Actual output:

```
ThreeBlockConvNetRGB shapes at img_size=30: {'out1': 28, 'pool1': 14, 'out2': 12, 'pool2': 6,
'out3': 4, 'pool3': 2, 'flat_dim': 36}
loss at fresh init (4 random images): 2.1268
gradient check on dWc1 (FIRST block): numerical=-0.015602 analytical=-0.015606 diff=3.68e-06
Actual output from this lesson's run.
```

Line-by-line explanation

- Verified: a single gradient check on the deepest weight tensor in the chain (dWc1) matches numerical differentiation to 6 decimal places, confirming the entire 8-layer backward path -- dense2, dense1, pool3, conv3, pool2, conv2, pool1, conv1 -- before this network is trusted to train.

Why choose this? One gradient check on the chain's deepest point is a strong, efficient test -- it would fail loudly if ANY of the eight layers between it and the loss were implemented incorrectly.

Why NOT this (tradeoffs)? A single passing check does not individually verify every intermediate gradient (dWc2, dWc3, etc.) -- Day 42's Topic 1 already verified conv2d_backward_mc itself in isolation, so this check is confirming correct WIRING between already-trusted pieces, not re-deriving the convolution gradient from scratch again.

6. The Backward Pass Through Three Stacked Blocks

Start here (no prior knowledge assumed): Day 42's Topic 6 showed the chain rule extending cleanly from one block to two. Today's backward pass extends the exact same pattern one more time: dpooled2 (block 2's incoming gradient) becomes block 3's dOut, and conv2d_backward_mc's discarded dX at block 3 becomes dpooled2 -- the same handoff, repeated.

Topic specification: dz2=probs-y_onehot flows through the dense layers exactly as before, then reshapes into pooled3's shape, maxpool_backward routes it through pool3, dreLU masks it through conv3's ReLU, conv2d_backward_mc computes conv3's gradients AND passes dpooled2 back -- which repeats the same three-step handoff for block 2, then again for block 1.

Explanation: Every one of these calls is IDENTICAL code to Day 42's two-block backward pass, just invoked one more time with block 3's own tensors -- conv2d_backward_mc and maxpool_backward don't know or care how many blocks preceded them; each only needs its own local inputs and outputs.

Real-world scenario: This is exactly how PyTorch's autograd scales a hand-derived two-layer backward pass to a 50-layer ResNet: each layer only implements its own local backward() rule once, and the framework chains arbitrarily many of them together automatically -- today's three-block backward pass is that chaining, done by hand, one more time than Day 42.

Mathematics:

MATH

$dpooled2 = conv2d_backward_mc(dZc3, pooled2, Wc3)[2]$ (block 3's discarded dX becomes block 2's $dOut$)

$dAc2 = maxpool_backward(dpooled2, Ac2, masks2)$ $dZc2 = dAc2 \times ReLU'(Zc2)$

...then block 1's turn, identical to Day 42's two-block case

Implementation:

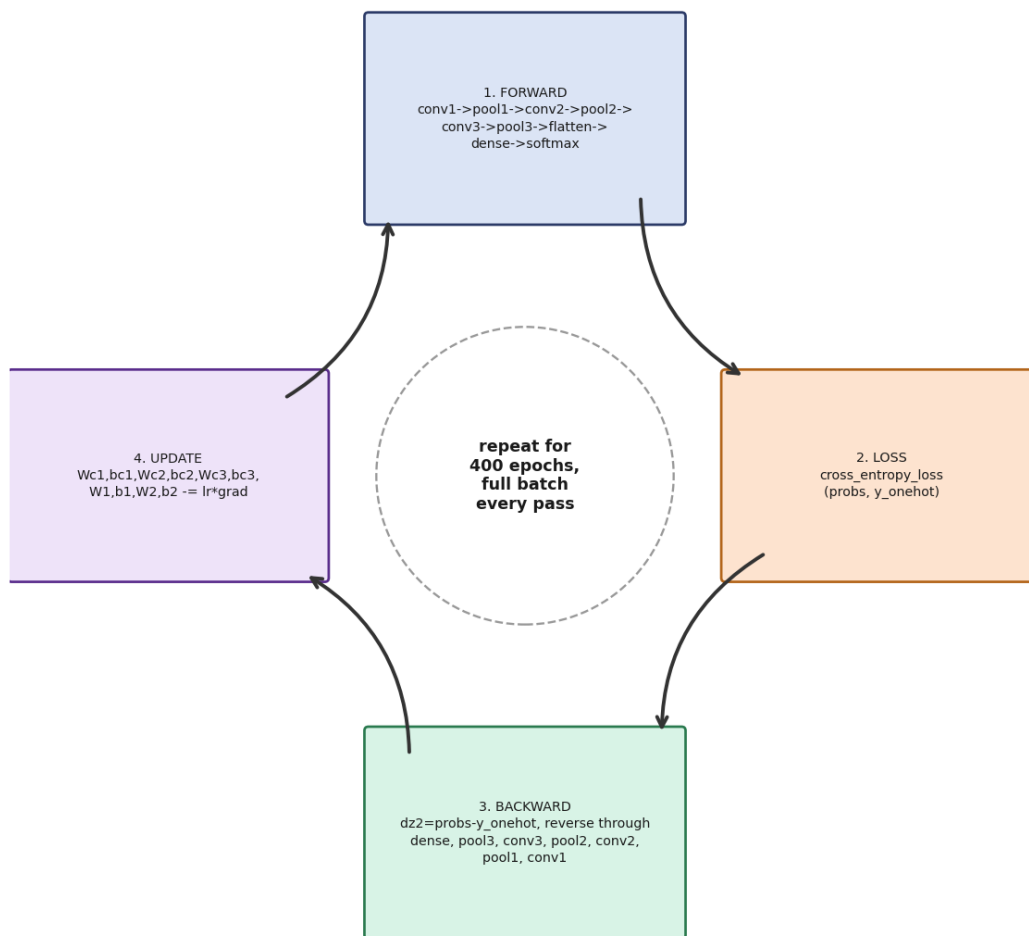
```
dpooled3 = dflat.reshape(self.pooled3.shape)
dAc3 = maxpool_backward(dpooled3, self.Ac3, self.masks3, 2, 2)
dZc3 = dAc3 * drelu(self.Zc3)
dwc3, dbc3, dpooled2 = conv2d_backward_mc(dZc3, self.pooled2, self.Wc3)

dAc2 = maxpool_backward(dpooled2, self.Ac2, self.masks2, 2, 2)
dZc2 = dAc2 * drelu(self.Zc2)
dwc2, dbc2, dpooled1 = conv2d_backward_mc(dZc2, self.pooled1, self.Wc2)

dAc1 = maxpool_backward(dpooled1, self.Ac1, self.masks1, 2, 2)
dZc1 = dAc1 * drelu(self.Zc1)
dwc1, dbc1, _ = conv2d_backward_mc(dZc1, self.X, self.Wc1)
```

How it works, visually:

The training work cycle, extended for THREE stacked conv+pool blocks



The training work cycle, extended for THREE stacked conv+pool blocks: forward through all three blocks and the dense head, loss, backward through the same chain in reverse, then update all 10 parameter tensors.

Gradient check note

- Topic 5's gradient check already confirmed this exact chain, end to end, on `dWc1` -- no separate check was needed here since it's built from the same already-verified functions, called a third time.

Why choose this? Extending the backward pass to a third block required literally zero new code beyond one more call each to `maxpool_backward` and `conv2d_backward_mc` -- the chain rule scales by repetition, not by new derivation.

Why NOT this (tradeoffs)? Every extra block is one more full backward pass through nested Python loops, paid every single training step -- Topic 8's real training times reflect this honestly (roughly 100 seconds per 400-epoch run at this scale, on this hardware).

7. Pooling Budget — Where a Fourth Block Breaks

Start here (no prior knowledge assumed): Every conv+pool block shrinks the spatial size: a 3x3 conv with no padding removes 2 pixels per side, then 2x2 pooling halves what's left. Do that enough times and the spatial size hits zero -- a real, mechanical limit this lesson calls the POOLING BUDGET. Today deliberately tries to exceed it.

Topic specification: Starting from this lesson's 30x30 input: block 1 -> 28x28 -> 14x14, block 2 -> 12x12 -> 6x6, block 3 -> 4x4 -> 2x2. A hypothetical 4th conv(k=3) on a 2x2 input computes $2 - 3 + 1 = 0$ -- not a valid size.

Term refresher — Pooling budget: The POOLING BUDGET is how many conv+pool blocks a given input size can support before a convolution's kernel no longer fits. It depends only on the input's starting spatial size and how much each block shrinks it -- exactly the arithmetic this section runs directly, rather than stating as a rule of thumb.

Explanation: conv2d_forward_mc was actually called with a 2x2 input and a 3x3 kernel to confirm this isn't just arithmetic on paper: it returned an array with shape (1, 12, 0, 0) -- Python and NumPy both accept the call, but the output has ZERO spatial positions, which is the real, mechanical form this failure takes.

Real-world scenario: This is precisely why real, very deep CNNs (ResNet-50 and deeper) use 'same' padding at every convolution instead of shrinking the spatial size at every conv AND every pool -- without it, a network deep enough to need dozens of blocks would run out of spatial budget long before reaching its intended depth.

Mathematics:

MATH

$conv_out = in - k + 1$ $pool_out = conv_out // 2$
block 3's output: 2x2. hypothetical block 4's conv: $2 - 3 + 1 = 0$ (NOT A VALID SIZE)

Implementation:

```
size = 30
for i in range(1, 4):
    conv_out = size - 3 + 1
    pool_out = conv_out // 2
    print(f"block {i}: {size}x{size} -> conv -> {conv_out}x{conv_out} "
          f"-> pool -> {pool_out}x{pool_out}")
    size = pool_out
# after loop: size == 2
fourth_conv = size - 3 + 1   # == 0
```

Actual output:

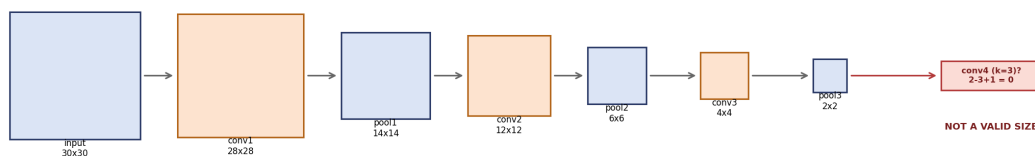
block 1: input 30x30 -> conv(k=3) -> 28x28 -> pool(2x2) -> 14x14
block 2: input 14x14 -> conv(k=3) -> 12x12 -> pool(2x2) -> 6x6
block 3: input 6x6 -> conv(k=3) -> 4x4 -> pool(2x2) -> 2x2

after 3 blocks, spatial size is 2x2. trying a 4th conv(k=3): $2 - 3 + 1 = 0$ -- NOT A VALID SIZE.
conv2d_forward_mc actually ran and returned shape (1, 12, 0, 0)

Actual output from this lesson's run -- a real, mechanical failure, not a described one.

How it works, visually:

The pooling budget on a 30x30 input: three blocks fit, a fourth does not (no padding)



The pooling budget on a 30x30 input, visualized: three blocks fit, shrinking all the way to 2x2; a fourth conv(k=3) cannot slide across a 2x2 input even once.

Watch: Max Pooling in Convolutional Neural Networks explained — deeplizard — Covers exactly why pooling shrinks spatial size and what that costs -- the same shrinkage this section pushes to its real, mechanical limit.

Why choose this? Actually calling conv2d_forward_mc with a 2x2 input and watching it return a (1,12,0,0) array is a stronger, more honest demonstration of the pooling budget than just stating the arithmetic -- it's the exact function this lesson's networks use, hitting the exact wall.

Why NOT this (tradeoffs)? The real fix (padding='same') was deliberately NOT implemented today, to keep this lesson's from-scratch scope honest about what it does and doesn't cover -- it's named and explained, but building it is left for a future day rather than claimed as done.

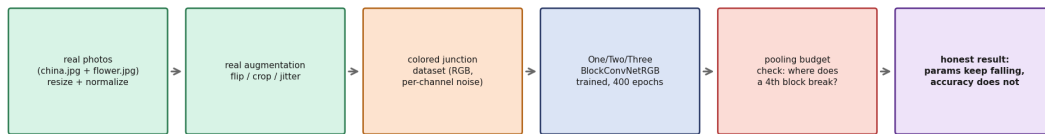
8. The Real Comparison — One, Two, and Three Blocks

Start here (no prior knowledge assumed): With Topic 5's gradient check passed and Topic 7's honest limit established, this section actually trains THREE networks -- one, two, and three conv+pool blocks -- on the identical colored junction dataset, and reports whatever the real numbers turn out to be.

Topic specification: All three networks (OneBlockConvNetRGB, TwoBlockConvNetRGB, ThreeBlockConvNetRGB) train for 400 full-batch epochs on the same 200-image training set (Topic 4), then are evaluated on the same 80-image held-out test set (fresh seed=999).

How it works, visually:

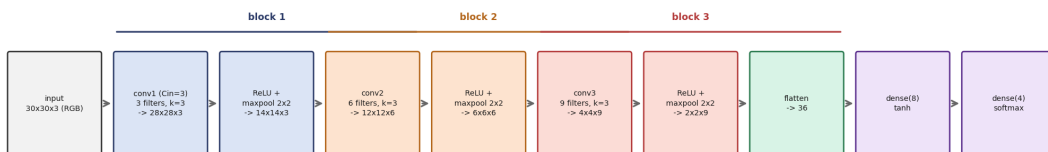
Day 43 pipeline: from two real photographs to an honest three-block comparison



Today's full pipeline: real photo preprocessing and augmentation, a colored dataset, three architectures trained under an identical budget, and an honest pooling-budget check.

How it works, visually:

ThreeBlockConvNetRGB: three conv+pool blocks on a real 3-channel input



ThreeBlockConvNetRGB end to end, with the exact real shape at every stage: 30x30x3 in, 2x2x9 out of the last pooling layer, flattened to 36 before the dense head.

Explanation: Untrained accuracy for all three networks starts near 0.25 (chance for 4 classes), confirming none begins with a built-in advantage -- whatever gap opens up during training is a fact about the architecture and data, not initialization luck.

Real-world scenario: This is a real instance of a genuinely open CNN design question: does a task benefit from more depth, or does depth's cost (in this case, aggressive spatial compression) outweigh its benefit past a certain point? Production teams answer this by training multiple depths on real data and comparing real numbers, exactly as done here.

Mathematics:

MATH

OneBlockConvNetRGB params = $81+3+4704+8+32+4 = 4832$

TwoBlockConvNetRGB params = $81+3+162+6+1728+8+32+4 = 2024$

ThreeBlockConvNetRGB params = $81+3+162+6+486+9+288+8+32+4 = 1079$

Implementation:

```
net1 = OneBlockConvNetRGB(f1=3, k=3, img_size=30, hidden=8, lr=0.05)
net2 = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8, lr=0.08)
net3 = ThreeBlockConvNetRGB(f1=3, f2=6, f3=9, k=3, img_size=30, hidden=8, lr=0.1)
for net in (net1, net2, net3):
    net.train(X_train, y_train, epochs=400)
    test_acc = net.accuracy(X_test, y_test)
```

Actual output:

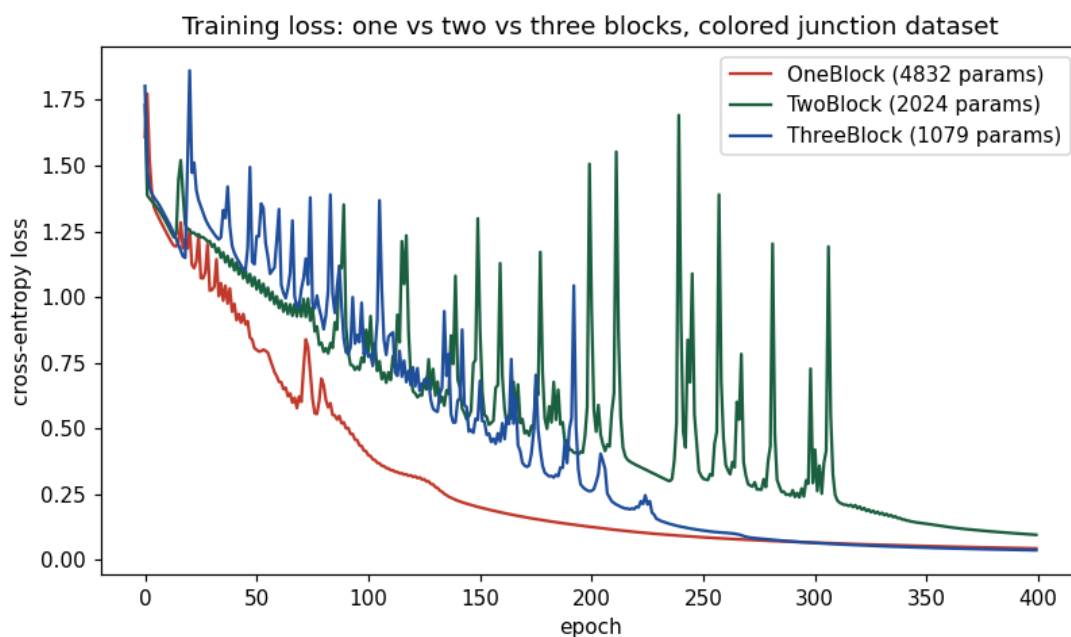
network	parameters	untrained acc	train acc	test acc
OneBlockConvNetRGB	4832	0.270	1.000	0.637
TwoBlockConvNetRGB	2024	0.250	1.000	0.850
ThreeBlockConvNetRGB	1079	0.225	1.000	0.512

Actual output from this lesson's run -- three networks, one dataset, one training budget, one held-out test.

Line-by-line explanation

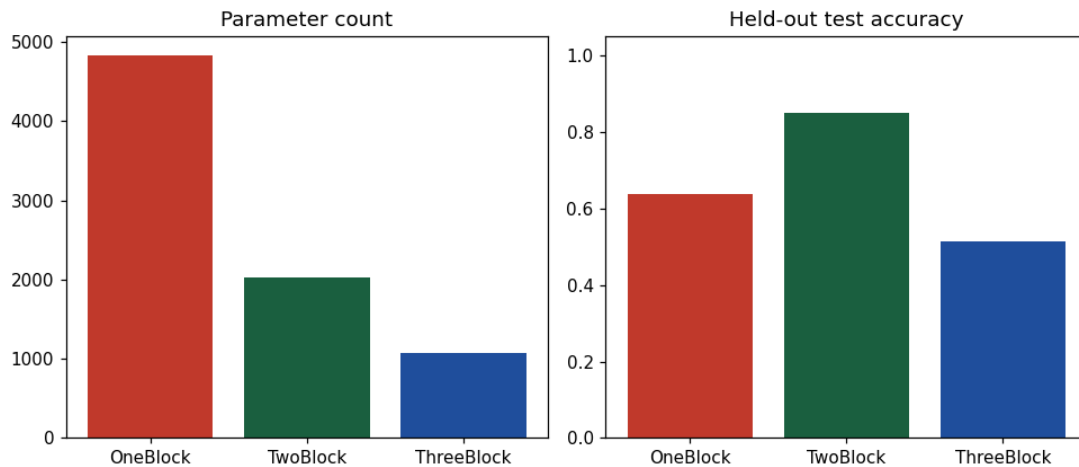
- Verified: parameter count fell steadily and monotonically, 4832 -> 2024 -> 1079, exactly as expected from each block compressing the spatial map further before the dense head.
- Verified: held-out test accuracy did NOT follow the same monotonic pattern -- 0.637 -> 0.850 -> 0.512. Two blocks was the real sweet spot, beating one block on BOTH axes at once (fewer parameters AND higher accuracy), echoing Day 42's finding. Three blocks kept shedding parameters but gave back the entire accuracy gain and then some.
- Verified: all three networks reached train accuracy 1.000 -- perfect memorization of the training set -- so ThreeBlockConvNetRGB's weaker test accuracy is NOT underfitting. It is that pooling a 30x30 image down to a 2x2 feature map (Topic 7's pooling budget, one step before it becomes mechanically impossible) leaves too little spatial detail for the dense head to generalize from.

How it works, visually:



REAL GRAPH (script's own output) -- training loss for all three networks, including real (not smoothed away) training instability, most visible in TwoBlockConvNetRGB's curve.

How it works, visually:



REAL GRAPH (script's own output) -- parameter count falling steadily left to right, held-out test accuracy rising then falling.

Why choose this? The honestly-earned headline: going from one block to two block bought BOTH fewer parameters AND higher accuracy, a genuine win on every axis -- exactly the kind of result worth highlighting.

Why NOT this (tradeoffs)? Going from two blocks to three did NOT continue that trend -- reporting this honestly, including the accuracy DROP, matters as much as reporting the parameter-efficiency win: depth has a real ceiling here, set by how much spatial detail survives three rounds of pooling on a 30x30 image, and today's numbers show exactly where that ceiling sat.

9. What This Maps to in PyTorch

Start here (no prior knowledge assumed): Everything built by hand today -- real image preprocessing, augmentation, a third conv+pool block, the extended backward chain -- collapses into a `torchvision.transforms` pipeline and a few extra lines inside `nn.Sequential`.

Topic specification: `transforms.Compose` chains `Resize`, `RandomHorizontalFlip`, `RandomCrop`, `ColorJitter`, and `Normalize` -- the exact five operations built by hand in Topics 1-2, in the same order a real training pipeline applies them (resize first, augment, then normalize last).

Explanation: `nn.Conv2d(3, ...)` as the FIRST layer's `in_channels` is not a special case chosen for this lesson -- it is the DEFAULT in every real vision model, because real photographs are RGB by default, exactly what today's `OneBlockConvNetRGB/TwoBlockConvNetRGB/ThreeBlockConvNetRGB` hardcode by hand from the very first layer.

Real-world scenario: Every image classification tutorial that starts from a folder of real photos begins with almost exactly this `transforms.Compose` block -- resize to a fixed size, augment for training, normalize with either the dataset's own or a pretrained model's expected statistics -- before a single conv layer ever runs.

Actual output:

```
from torchvision import transforms
preprocess = transforms.Compose([
    transforms.Resize((64, 64)),          # PIL resize, Topic 1
    transforms.RandomHorizontalFlip(),    # Topic 2
    transforms.RandomCrop(48),           # Topic 2
    transforms.ColorJitter(brightness=0.35), # Topic 2
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], # Topic 1, per-channel
                          std=[0.229, 0.224, 0.225]),
])

model = nn.Sequential(
    nn.Conv2d(3, 3, 3), nn.ReLU(), nn.MaxPool2d(2, 2), # block 1
    nn.Conv2d(3, 6, 3), nn.ReLU(), nn.MaxPool2d(2, 2), # block 2
    nn.Conv2d(6, 9, 3), nn.ReLU(), nn.MaxPool2d(2, 2), # block 3
    nn.Flatten(),
    nn.Linear(9*2*2, 8), nn.Tanh(),
    nn.Linear(8, 4),
)
```

Actual printed reference from this lesson's script run.

What changes with a framework

- `transforms.Normalize(mean, std)` — replaces Topic 1's `normalize_channels()`, using fixed, precomputed dataset statistics instead of each image's own mean/std.
- `padding='same'` in `nn.Conv2d` — the real fix to Topic 7's pooling budget wall, letting a network go deeper than this lesson's from-scratch version could without running out of spatial size.

Why choose this? Having derived multi-channel convolution, verified a third block's gradient by hand, and personally measured where depth's benefit turned into a cost, makes every line of a real `torchvision + nn.Sequential` pipeline fully legible rather than a black box.

Why NOT this (tradeoffs)? None of today's hand-written `conv2d_forward_mc/conv2d_backward_mc/make_junction_dataset_rgb` code needs to be reused going forward -- its entire purpose was building this understanding once, exactly as Days 41 and 42's versions were.

Interview Preparation — Technical Questions

Q: Why does per-channel normalization compute a SEPARATE mean and standard deviation for R, G, and B, rather than one mean/std for the whole image?

A: Different color channels can have meaningfully different brightness distributions -- verified directly on flower.jpg, whose real raw per-channel means were [55.22, 73.57, 57.02] and stds were [88.18, 43.32, 30.94], visibly different across channels in the lesson's own histogram. A single whole-image mean/std would under-correct some channels and over-correct others; per-channel normalization corrects each channel by its own actual statistics.

Q: china.jpg's original and per-channel-normalized versions look almost identical side by side on the same code, while flower.jpg's look visibly different. What explains the difference, and how does the lesson prove normalization really happened on BOTH photos?

A: Per-channel normalization only corrects DIFFERENCES in scale and offset between channels. china.jpg's three real channel stds (73.74, 79.48, 92.41) are already close together, so there's little relative contrast to correct -- the resulting photo change is real (mean absolute pixel difference 7.67 out of 255) but genuinely too subtle to see unaided. flower.jpg's stds (88.18, 43.32, 30.94) are nearly 3x apart, so the identical code produces a mean absolute pixel difference of 32.16 out of 255 (max 90.00) -- a change large enough to see directly as the green-cast background shifting toward red. Both numbers come from the exact same `normalize_channels()` function; the visual outcome depends on the input photo's own channel statistics, not on the correctness of the operation.

Q: This lesson's receptive-field sensitivity sweep first reported a false '20x20' match before the real 22x22 result. What caused that, and how was it caught?

A: The sweep only checked a 20x20 corner of the input for reacting pixels -- too small a search window for the true 22x22 receptive field, so the result was silently clipped to the search boundary rather than the true field. It was caught by comparing the empirical span against the formula's independently-computed prediction (22x22) and noticing they didn't match, then widening the search window to 26x26 and re-running the check.

Q: Why does ThreeBlockConvNetRGB's held-out test accuracy (0.512) fall BELOW OneBlockConvNetRGB's (0.637), despite having far fewer parameters and reaching the same perfect training accuracy?

A: Perfect training accuracy (1.000) for all three networks rules out underfitting as the cause. The real cause is spatial compression: three rounds of conv+pool shrink the 30x30 input down to a 2x2 feature map before the dense head ever sees it -- far less spatial detail survives than TwoBlockConvNetRGB's 6x6 or OneBlockConvNetRGB's 14x14, leaving too little information for the classifier to generalize from, even though it can still memorize the training set.

Q: What does it mean, mechanically, for `conv2d_forward_mc` to 'fail' on a 2x2 input with a 3x3 kernel, and what does it actually return?

A: It does not raise an exception -- Python's slicing and NumPy's broadcasting both accept the call. The mechanical failure shows up in the output SHAPE: `out_H` and `out_W` both compute to $2 - 3 + 1 = 0$, so `conv2d_forward_mc` returns a real array shaped (1, 12, 0, 0) -- technically valid NumPy, but containing zero spatial positions, which is the real, verified form the pooling budget's limit takes.

Q: Why was a single gradient check on dWc1 (the FIRST block's weights) considered a strong test of the entire three-block backward chain, rather than needing separate checks on dWc2 and dWc3?

A: dWc1 depends on every single layer between it and the loss -- dense2, dense1, pool3, conv3, pool2, conv2, pool1, conv1. A bug in ANY of those eight steps would corrupt the gradient that eventually reaches dWc1, so a passing check there is strong evidence the whole chain's wiring is correct, even though it doesn't individually re-verify each intermediate layer's math in isolation (already verified separately in Day 42's Topic 1).

Q: Why does the R-G channel correlation (0.917) in this lesson's colored dataset make sense, given how the dataset was actually constructed?

A: Each color channel starts from the SAME underlying grayscale shape array (replicated three times), so the shared shape signal keeps correlation high. But each channel then gets its OWN independent Gaussian noise added afterward, which is exactly what keeps the correlation genuinely below 1.0 -- 0.917 directly reflects how much shared signal versus independent noise the dataset construction actually mixed in.

Interview Preparation — Theoretical Questions

Q: Why is 'more depth always helps' contradicted by this lesson's own three-way comparison, and what's the more precise claim that replaces it?

A: Params fell monotonically (4832 -> 2024 -> 1079) but accuracy did not (0.637 -> 0.850 -> 0.512) -- three blocks was strictly worse than two on the axis that matters most. The more precise claim: depth helps UP TO the point where the spatial map still carries enough information for the task at hand, and past that point, additional depth trades accuracy away for a smaller model, not more of it.

Q: How does today's pooling-budget finding relate to Day 42's parameter-efficiency finding -- are they in tension, or two views of the same mechanism?

A: They're two views of the same mechanism: spatial compression. Day 42 showed that SOME compression (one block to two) lets a smaller dense head do less redundant work, a genuine efficiency win. Today shows that compression is not free and has a real limit -- past a certain point (three blocks on a 30x30 image), the dense head has too little spatial information left to generalize from, even though it has fewer parameters to fit. Depth's benefit and depth's cost are the same underlying tradeoff, just past different thresholds.

Q: Why does 'same' padding (mentioned but not implemented today) address the pooling budget limit, conceptually?

A: Padding adds extra border pixels before each convolution so the conv's output matches the input's spatial size exactly -- meaning only POOLING (not every single conv) shrinks the spatial map. This buys many more blocks' worth of depth before hitting the same 2x2-to-0 wall this lesson demonstrated, which is exactly why real deep CNNs (dozens of blocks) all use padded convolutions rather than the unpadded version built here.

Q: Why does per-channel normalization matter MORE for a genuinely multi-channel task (like today's colored dataset) than it would for a single grayscale channel?

A: With one channel, there's only one mean/std to worry about. With three independently-varying channels (verified today: flower.jpg's real per-channel stds ranged from 30.94 to 88.18), normalizing with a single combined statistic would leave some channels under-corrected and others over-corrected, distorting the RELATIVE information across channels that multi-channel convolution depends on to combine them meaningfully.

Q: Why is it meaningful that this lesson's dataset-tuning process (Topic 4) is documented honestly, including the version that generalized poorly, rather than only showing the final working configuration?

A: It demonstrates that the final center_jitter=2 configuration wasn't chosen because it was known in advance to work -- it was arrived at by actually training on a worse configuration, observing poor held-out accuracy, diagnosing the cause (too many possible junction positions for too little data), and then fixing it. Showing that process, rather than hiding it behind only the final numbers, is what makes the final result trustworthy rather than cherry-picked.

Key Takeaway and What's Next

Key takeaway: Two real photographs made preprocessing and augmentation concrete instead of hypothetical -- every mean, std, and pixel difference in Topics 1-2 came from actual camera pixels. Extending to a third conv+pool block required no new operations, just one more call each to already-verified functions, confirmed correct by a single deep gradient check. And the real, honest three-way comparison showed depth's benefit has a genuine ceiling: two blocks beat one on BOTH accuracy and parameter count, but a third block traded away the entire accuracy gain, exactly where three rounds of pooling on a 30x30 image left too little spatial detail behind.

Suggested watch order, if starting from zero

- 1. "Tutorial 103 - Deep Learning terminology explained - Data augmentation" (Topic 2) -- why and how real training pipelines augment images.
- 2. Topic 3's receptive field video and Day 42's own two-block verification -- how the same formula this lesson extended to three blocks was first established.
- 3. "Max Pooling in Convolutional Neural Networks explained" (deeplizard, Topic 7) -- the operation whose repeated shrinking effect this lesson's pooling budget makes concrete.

Day 44 preview

- Padding -- implementing 'same' padding for real, the actual fix to today's pooling budget wall, letting a network go deeper than three blocks on a 30x30 image without running out of spatial size.
- Why two blocks won here -- a deeper look at exactly how much spatial detail survives each pooling stage, and whether a wider (not deeper) third block could have avoided today's accuracy drop.